BAMBINOS.LIVE

AI Program Manager (Operations) — Take-Home Case

Re-engineering the Student Renewal → Class Scheduling Pipeline

A deterministic reconciliation engine, exception operating model and n8n automation blueprint

Executive recommendation

Replace the manual teacher form → Ops verification → booking request handoffs with an event-driven renewal orchestration layer. Verify gateway settlement and canonical student/batch data deterministically; fast-track only clean same-batch renewals; route every discrepancy to a named owner with an SLA; and keep actual schedule creation idempotent and capacity-aware. AI should draft communications, not decide payment validity, identity or child placement.

Assessment evidence

The official masked sample includes 13 invoices, 12 faculty forms, 4 booking requests, 2 schedule-booking rows and 13 active-reference rows. Running the included prototype produced the following decisions:

Metric	Result
Auto-approval ready	3
Reason-coded exceptions	12
Open booking requests	3
Paid renewal invoices without a form	3 (included in exceptions)

These are counts from this assessment snapshot only, not company-wide rates or measured time savings. “Auto-approval ready” means eligible to create a booking request after an idempotency check; it does not mean future class dates were booked.

A. Root-cause problem definition & leakage audit

Where the pipeline leaks

• Invoices → faculty form: teacher delay, wrong invoice ID/link, duplicate submissions after browser errors, form contact not matching payer, and renewals with no form.

• Faculty form → booking request: manual Ops queue, repeated payment checks, amount/discount confusion (gross amount vs net amount), incorrect student/course/educator IDs, weekend/after-hours gaps and unclear ownership.

• Booking request → schedule: same-batch requests still wait for allocation; batch is full; parent requests a different batch; schedule availability/IDs are missing; a request can remain pending after payment.

• Cross-table risks: duplicate booking requests, stale current-batch references, soft-deleted records, partial/failed payment mistaken for paid, masked PII that cannot safely support fuzzy matching, and schedule rows that represent only some purchased sessions.

Observed sample exceptions

• 84904: form mobile differs from invoice mobile → PARENT_CONTACT_MISMATCH; human guardian verification, not fuzzy matching.

• 84905: same batch is at 6/6 capacity and request 5505 is Failed_Batch_Full → capacity exception and scheduling escalation.

• 84908: two faculty forms (3308/3309) for one invoice → duplicate form; process one canonical record.

• 84909 / 84911: Partially_Paid / Failed → no approval until gateway confirms full settlement.

• 84912: continue_same_batch=No and request 5512 awaits parent selection → batch-change workflow.

• Form 3313 points to invoice 84931, while invoice 84913 exists → invoice not found plus a paid invoice without a matched form.

• 84906, 84910, 84913: paid renewal invoices have no matched faculty form → teacher reminder and escalation.

Leakage metrics and proposed SLAs

Metric	Definition	Initial target
Orphaned paid invoice rate	Paid renewal invoices with no valid form ÷ paid renewal invoices	< 1%; alert at 12h
Payment-to-booking-request p95	Paid timestamp to booking request creation	< 30 min for clean same-batch
Paid-to-first-class p95	Paid timestamp to first confirmed class	< 24h or before next due class
Duplicate form rate	Duplicate forms per invoice	< 0.2%; block duplicate writes
Exception ageing	Time since exception created	Owner within 15 min; resolve < 4 business hours
Batch allocation failure	Requests not allocated due to capacity	100% alternate-batch offer within 2h
Schedule completeness	Purchased sessions with valid schedule rows	100% before marking schedule complete

B. Re-engineered zero-touch workflow

Target architecture

Payment gateway webhook → n8n event intake → canonical invoice/student/batch lookup → deterministic rules engine → (a) same-batch, fully paid, capacity-safe: create booking request idempotently and trigger schedule allocator; (b) exception: create case with reason code, owner, age timer and message draft → human resolution → re-run validation → booking. A daily reconciliation sweep catches missed webhooks and orphaned records.

• Remove the teacher renewal form for the happy path: payment metadata already identifies student, course and teacher; use the authenticated student record and renewal intent captured at checkout. Retain a teacher correction form only for missing/changed data.

• Eliminate manual Ops payment verification by querying the gateway API and validating captured/settled state, currency, invoice ID and net amount. Never trust a screenshot or client-side form as payment proof.

• For same-batch renewal, reuse canonical batch ID and schedule template only when capacity and schedule policy pass. Use a unique idempotency key (invoice_id + renewal_term) to prevent duplicate requests.

• Use an LLM only to classify free-text remarks into a suggested queue and draft messages. Structured rule outcomes remain authoritative; low-confidence or conflicting data always goes to a human.

Exception routing matrix

Reason code	Primary owner	Action
PAYMENT_FAILED / PARTIAL_PAYMENT	Finance Ops	Verify gateway capture; collect balance if due; re-run
PARENT_CONTACT_MISMATCH	Parent Support	Verify guardian through existing trusted channel
AMOUNT_MISMATCH	Finance + Teacher	Compare invoice amount, discount and net paid
DUPLICATE_FORM / REQUEST_EXISTS	Ops	Deduplicate; continue canonical request
BATCH_AT_CAPACITY	Scheduling Ops	Find eligible alternative; obtain parent consent

Reason code	Primary owner	Action
BATCH_CHANGE_REQUEST	Scheduling Ops	Offer slots; parent selects; confirm teacher/capacity
INVOICE_NOT_FOUND	Teacher + Ops	Correct invoice reference and verify gateway record
PAID_INVOICE_NO_FORM	Teacher → Faculty Lead	Auto-remind; escalate after 12h

C. Working prototype & n8n design

The repository includes a runnable Python reconciliation engine and an importable n8n workflow. The n8n workflow accepts five JSON arrays by webhook, runs deterministic checks in a Code node and returns auto-approval-ready rows, exceptions, message drafts and booking backlog. The local script reads the five CSVs and writes matching result files.

• Prototype result: 3 auto-approval-ready rows (84903, 84907, 84908); 12 exceptions; 3 open booking requests. The 84908 duplicate form is separately flagged and is not a second approval.

• Discount logic compares faculty amount_paid to invoice net_amount, not gross amount. Thus 84907 correctly reconciles ■7,600 after ■1,900 discount against ■7,600 paid.

• No schedule writes or WhatsApp sends are enabled. Before production, add authenticated gateway lookup, DB transaction, unique constraints, retry/dead-letter queue, secrets management, audit log and approval gates.

D. Ops dashboard, SOP & rollout

Daily Ops Head dashboard

• Top cards: paid renewals today; auto-approved; stuck invoices >12h; forms awaiting validation; booking requests awaiting allocation; first-class scheduled; exception backlog; hours saved (measured against baseline).

• Funnel: paid → form/intent captured → verified → booking request → batch allocated → schedule rows created → first class attended. Show counts and conversion at each transition.

• Exception table: invoice/student masked ID, reason code, owner, created age, next action, SLA breach indicator, last parent/teacher contact. Filters by region/timezone, course, teacher, payment gateway and renewal date.

• Capacity panel: batch occupancy, available seats, failed allocations, alternative slots offered and accepted.

Operating SOP

• 09:00 local Ops review: review breached SLAs, payment failures, >12h orphan invoices and capacity exceptions.

• Every exception has one owner, one reason code, one next action and a due time. Reassignment is logged.

• Before schedule creation: re-check payment, active student, batch capacity and duplicate key inside one transaction.

• After creation: verify schedule row count against purchased sessions and send parent confirmation only after successful persistence.

• If a schedule conflict occurs, stop new writes for that invoice, preserve existing classes, alert Scheduling Ops and use rollback/replay with idempotency.

30–60–90 day plan

Phase	Delivery	Exit criteria
Days 1–30: Instrument + shadow	Map production fields; baseline funnel/SLA; read-only gateway and DB lookups; shadow-mode reconciliation for every webhook	≥98% reconciliation accuracy; zero duplicate/incorrect auto-approvals in shadow mode
Days 31–60: Controlled automation	Enable auto-approval for clean same-batch renewals for 10–20% cohort; exception queue; daily ops review	≥95% clean cases straight-through; exception queue cleared daily; no duplicate schedules
Days 61–90: Scale + optimize	Expand by course/region/timezone; retire redundant form for eligible cases; dashboard; booking backlog; parent reminders	≥99% accuracy; ≥80% eligible cases auto-approved; booking backlog cleared weekly

Change management & risk controls

• Train 200+ teachers using 15-minute role-based demos, one-page exception guide and office hours; keep the legacy form available as fallback for 2 weeks.

• Ops team of 10 becomes exception-resolution and quality-control owners, not data-entry staff. Assign a daily on-call lead and a finance/scheduling escalation rota.

• Run shadow mode against existing manual approvals; compare every decision and investigate false positives before enabling writes. Roll out by cohort with kill switch and one-click pause.

• Protect children and family data: minimum necessary PII, role-based access, encrypted secrets, masked logs, retention policy and no LLM exposure of payment credentials.

Assumptions and limits

The assessment CSVs are masked and limited. Phone/email mismatch is treated as a hard exception because masked values cannot prove identity. Capacity counts are a snapshot and must be re-read at transaction time. The sample does not provide the full future schedule inventory or all 24 class dates, so the prototype prepares booking requests but does not fabricate class schedule rows. Targets above are proposed operating goals, not measured current performance.

Submission artifacts

README, five CSV inputs, Python reconciliation script, generated CSV/JSON results, exception-drafting prompt, and n8n workflow export are included alongside this report.